

What is an Agent?

A brief history of language modeling

Shreya Havaladar

Introductions



Shreya



Aaditya (TA)

A little about me!

- From LA + Orange County
- Graduated from USC in 2021!
- Got my PhD at Penn in May 2026

Researched sociotechnical alignment of LLMs (how personal + cultural context influence what a “correct generation” should be)



My mom + my dog!

Welcome!

A few things to note

1. This is the first time this class is being offered at Columbia
2. There will (inevitably) be some bumps in the road! We will work them out!!
3. This course will be equal parts computer science and design.

Welcome!

What you'll get from this course

1. A basic understanding of how LLMs work
2. Overview of agentic design principles
3. Insights into how agents are being used in academia + industry
4. Hands-on project where you build an agent!

Course website: <https://designing-agents.github.io/>

Expected difficulty

Full use of AI agents is allowed for coding components of HWs and the final project

- Main goal: learn how agents fit into your workflow, and practice orchestration and delegation

Because of this, **what you put in is what you get out**

- This course (likely) won't be stressful
- It's on to take charge of your own learning

Grading

Grades will be 45% homeworks, 35% final project, and 20% participation.

- HWs are individual
- Final project is in groups of 3
- Participation is based on attendance + contribution during class labs
 - You are allowed to miss class 2 times without penalty
- No exams!!

Is this course for me?

- **Yes! if you have...**
 - No ML experience
 - Some ML experience
 - Taken 1-2 ML courses, but feel you would benefit from a refresher
 - Not done too many applied AI projects
 - Curiosity about how agents work and are built!
 - ***Willingness to engage with and talk to your peers***

Is this course for me?

- **Probably not, if you have...**
 - Extensive ML experience
 - Taken multiple ML courses
 - Fully understand how LLMs are trained, improved, and deployed
 - Completed many applied AI projects
 - Built any type of agent yourself already
 - ***Prefer classes where attendance/participation is optional***

** Come talk to me after class if you're not sure*

Policies

- AI tools are allowed for coding tasks, provided **you fully understand and can explain any code you submit.**
- AI delegation is NOT allowed for writing components of assignments.
 - Short-answer questions in the HWs
 - Design document for final project
 - Note: collaboration != delegation
- You have 3 late days you can use as needed

Lecture Format

Class will (typically) consist of:

- Around 75 min of lecture
- Around 45 min of in-class labs / activities
 - *Proof of completion will serve as attendance*

I will do my best to get you out of here by 9pm!!

It is okay to eat dinner in class (as long as it's not disruptive)

Today's Lecture

1. **History of language modeling**
2. Defining the AI Agent

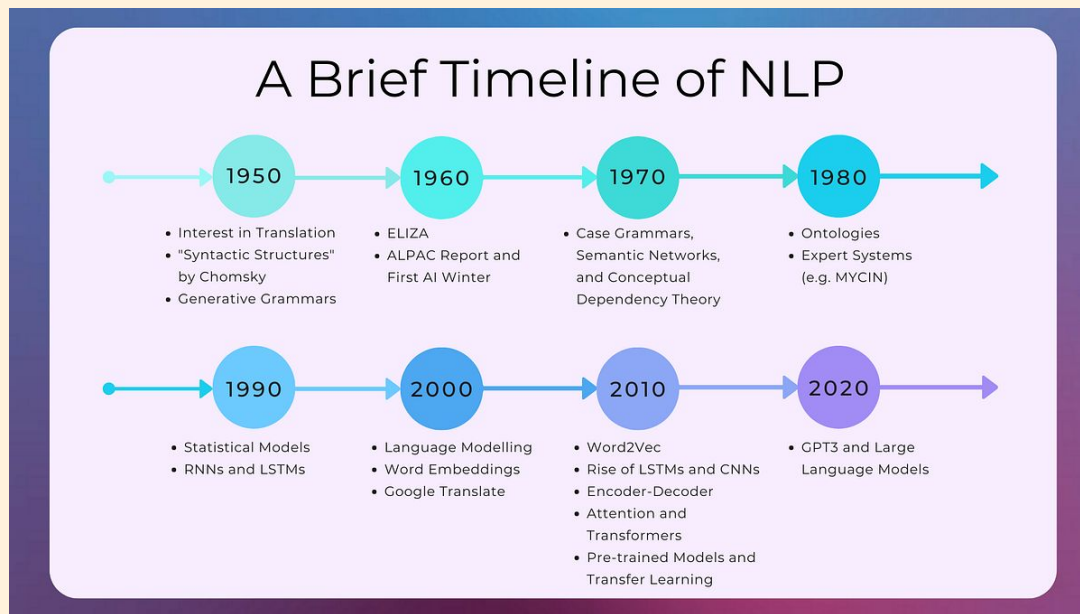
The Evolution of language models

A Brief Timeline of NLP



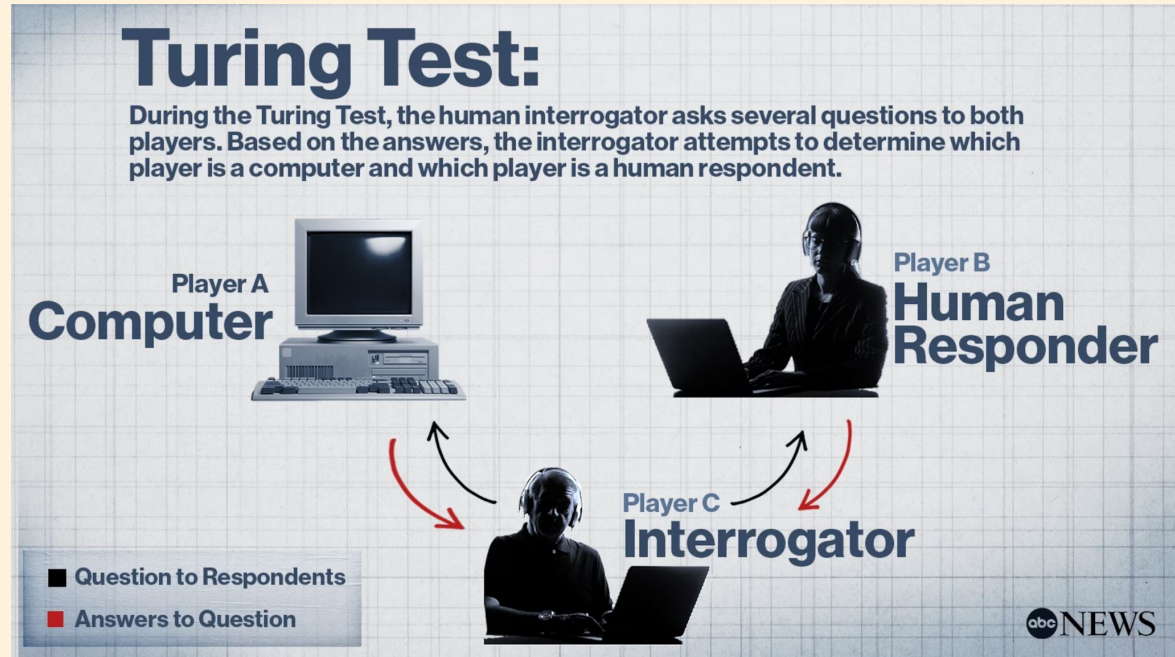
The Rule-Based Era (1950s – 1990s)

Early researchers viewed language as a set of logical puzzles. If you could map every grammatical rule, you could "solve" language.



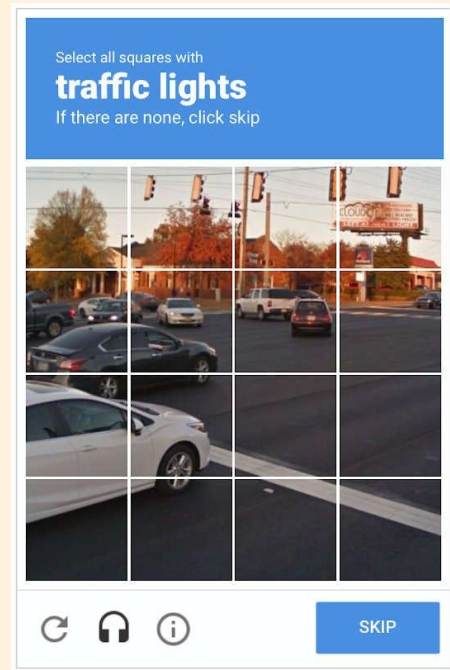
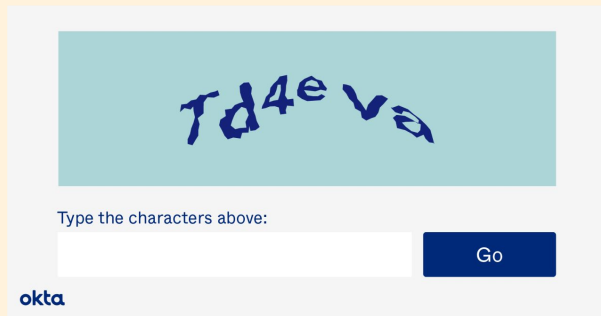
The Turing Test (1950)

Alan Turing proposed that if a machine could mimic human conversation perfectly, it could be considered "thinking."



The Turing Test (1950)

CAPTCHA (Completely Automated Public Turing test to tell Computers and Humans Apart)



Chomskyan Linguistics

Noam Chomsky argued for "Universal Grammar" — the idea that humans have an innate, rule-based biological structure for language.

Computers tried to mimic this with **Context-Free Grammars (CFGs)**.

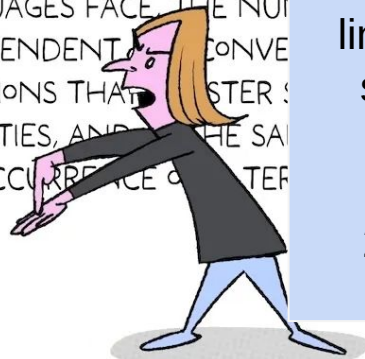


Chomskyan Linguistics

IT IS A WELL-KNOWN FACT THAT THE SCOPE OF NEGATION DOES NOT EXTEND ACROSS A SENTENCE BOUNDARY (UNLIKE SUCH PHENOMENA AS PRONOMINAL ANAPHORA). THUS, THE INTERPRETATION OF ELEMENTS LIKE NEGATION CAN ONLY BE GUIDED BY THE CORRECT ANALYSIS.



WE CAN USE THE NOTION OF GRAMMATICAL SUBJECT TO ILLUSTRATE THE MULTI-CONVENTIONAL PROBLEM THAT ALL NATURAL LANGUAGES FACE. THE NOUN PHRASES OF THESE LANGUAGES ARE INDEPENDENT OF THE CONVENTIONAL SOLUTIONS THAT OTHER LANGUAGES USE. THE PROPERTIES, AND THE SAME, OF THE OCCURRENCE OF OTHER SOLUTIONS.



Controversial viewpoint — other linguists believed there was no set of grammatical rules that

1. spanned all languages
2. humans inherently knew

CFGs

A formal system in computer science that uses recursive rules to describe the structure of languages.

```

<Stmt> → <Id> = <RExpr> ;
<Stmt> → { <StmtList> }
<Stmt> → if ( <RExpr> ) <Stmt>
<StmtList> → <Stmt>
<StmtList> → <StmtList> <Stmt>
<RExpr> → <RExpr> > <AExpr>
<RExpr> → <RExpr> < <AExpr>
<RExpr> → <RExpr> >= <AExpr>
<RExpr> → <RExpr> <= <AExpr>
<RExpr> → <AExpr>
<AExpr> → <AExpr> + <PExpr>
<AExpr> → <AExpr> - <PExpr>
<AExpr> → <PExpr>
<PExpr> → <Id>
<PExpr> → <Num>
<Id> → x
<Id> → y
<Num> → 0
<Num> → 1
<Num> → 9
    
```

```

                                <Stmt>
if ( <RExpr> ) <Stmt>
if ( <RExpr> > <AExpr> ) <Stmt>
if ( <AExpr> > <AExpr> ) <Stmt>
if ( <PExpr> > <PExpr> ) <Stmt>
if ( <Id> > <Num> ) <Stmt>
if ( x > 9 ) <Stmt>
if ( x > 9 ) { <StmtList> }
if ( x > 9 ) { <StmtList> <Stmt> }
if ( x > 9 ) { <Stmt> <Stmt> }
if ( x > 9 ) { <Id> = <RExpr> ; <Stmt> }
if ( x > 9 ) { x = <AExpr> ; <Stmt> }
if ( x > 9 ) { x = <PExpr> ; <Stmt> }
if ( x > 9 ) { x = <Num> ; <Stmt> }
if ( x > 9 ) { x = 0 ; <Stmt> }
if ( x > 9 ) { x = 0 ; <Id> = <RExpr> ; }
if ( x > 9 ) { x = 0 ; y = <AExpr> ; }
if ( x > 9 ) { x = 0 ; y = <AExpr> + <PExpr> ; }
if ( x > 9 ) { x = 0 ; y = <PExpr> + <PExpr> ; }
if ( x > 9 ) { x = 0 ; y = <Id> + <Num> ; }
if ( x > 9 ) { x = 0 ; y = y + 1 ; }
    
```

Activity: Rules for WSD

WSD = **Word Sense Disambiguation**

For example, does “date” mean

- (1) a dried fruit
- (2) a romantic meeting
- (3) a calendar day

Get into groups of 2!



Activity: Rules for WSD

Phase 1: Build Your “Date” Rules (20 Minutes)

Write strict rules **that ONLY using logic**. Examples:

IF "love" is in the sentence, THEN it is a romantic meeting.

IF "january" is in the sentence, THEN it is a calendar day.

```
def disambiguate_date(input_sentence):  
    ...  
    return {"dried_fruit", "romantic date", "calendar_day"}
```

Activity: Rules for WSD

Phase 2: Break someone else's "Date" Rules (10 Minutes)

IF "love" is in the sentence, THEN it is a romantic meeting.

IF "january" is in the sentence, THEN it is a calendar day.

→ *I have a Hinge **date** planned in **January***

→ *I **love** to put **dates** in my smoothie*

Feel free to use AI tools!

Activity: Rules for WSD

Submit the sentence you used to break your opponent's system on courseworks for attendance credit

Code: 6G4YC

The Statistical Shift (1990s – 2010s)

Key Question:

How do you model data where context matters over time?

The Statistical Shift (1990s – 2010s)

The world is too messy for rules!

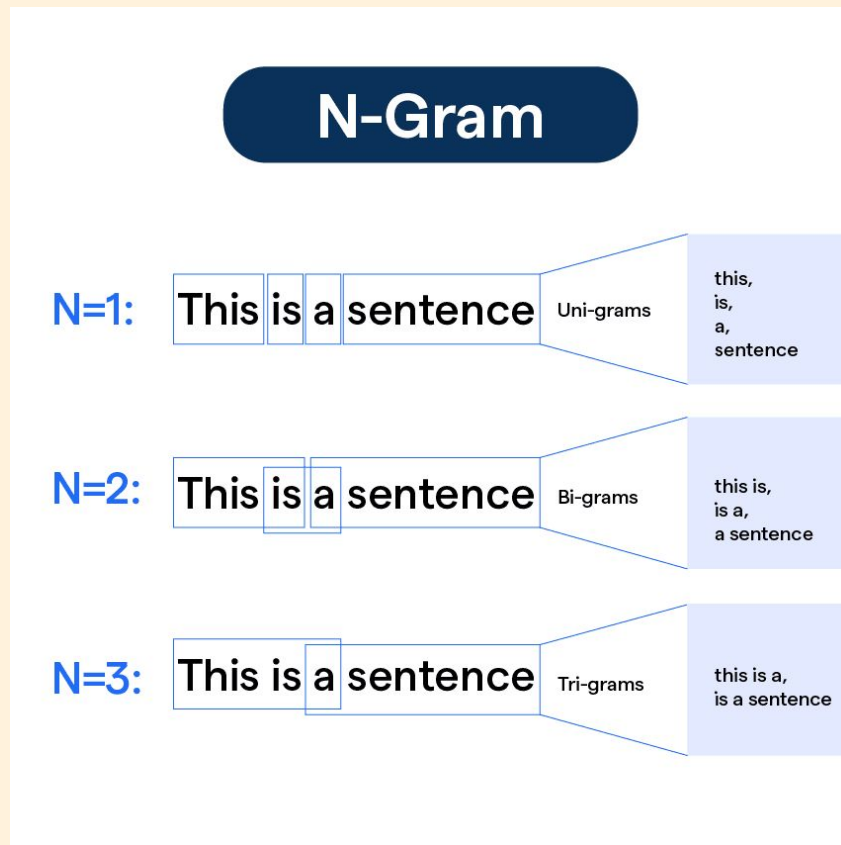
Instead of rules, researchers began using large corpora (like the Wall Street Journal, Wikipedia, etc.) to calculate the probability of word sequences.



N-Grams

Predicting the n^{th} word based on the previous $n-1$ words.

This is fundamentally a **Markov Chain** — the future state depends only on the current state, not the entire past history



N-Grams: an example

Corpus:

I like cats

I like dogs

I love cats

You like dogs

N-Grams: an example

Corpus:

I like cats

I like dogs

I love cats

You like dogs

Vocabulary:

{I, You, like, love, cats, dogs}

N-Grams: an example

Corpus:

I like cats

I like dogs

I love cats

You like dogs

Vocabulary:

{I, You, like, love, cats, dogs}

Create a bigram model
for this corpus:

$$P(w_t | w_{t-1})$$

N-Grams: an example

Corpus:

I like cats

I like dogs

I love cats

You like dogs

Current word	Next word	Count
I	like	2
I	love	
You	like	
like	cats	
like	dogs	
love	cats	

N-Grams: an example

Corpus:

I like cats

I like dogs

I love cats

You like dogs

Current word	Next word	Count
I	like	2
I	love	1
You	like	1
like	cats	1
like	dogs	2
love	cats	1

N-Grams: an example

Current word	Next word	Count
I	like	2
I	love	1
You	like	1
like	cats	1
like	dogs	2
love	cats	1

$$P(\textit{like} \mid \textit{I}) = \frac{2}{3}$$

$$P(\textit{love} \mid \textit{I}) = ?$$

$$P(\textit{dogs} \mid \textit{like}) = ?$$

$$P(\textit{cats} \mid \textit{like}) = ?$$

N-Grams: an example

Current word	Next word	Count
I	like	2
I	love	1
You	like	1
like	cats	1
like	dogs	2
love	cats	1

$$P(\textit{like} \mid \textit{I}) = \frac{2}{3}$$

$$P(\textit{love} \mid \textit{I}) = \frac{1}{3}$$

$$P(\textit{dogs} \mid \textit{like}) = \frac{2}{3}$$

$$P(\textit{cats} \mid \textit{like}) = \frac{1}{3}$$

N-Grams: an example

$$P(\textit{like} \mid I) = \frac{2}{3}$$

$$P(\textit{love} \mid I) = \frac{1}{3}$$

$$P(\textit{dogs} \mid \textit{like}) = \frac{2}{3}$$

$$P(\textit{cats} \mid \textit{like}) = \frac{1}{3}$$

What would this model generate with the Prompt:

“I ...”

N-Grams: an example

$$P(\textit{like} \mid I) = \frac{2}{3}$$

Would this model ever generate:

$$P(\textit{love} \mid I) = \frac{1}{3}$$

1. “I love dogs”?

$$P(\textit{dogs} \mid \textit{like}) = \frac{2}{3}$$

2. “I hate cats”?

$$P(\textit{cats} \mid \textit{like}) = \frac{1}{3}$$

Deep Learning (2010s)

- Motivation: learn representations so **similar words and contexts can share information**
 - Not possible with purely statistical methods like n-gram modeling
- Increased availability of GPUs → deeper, multi-layer architectures

Deep Learning (2010s)

- 2012 AlexNet breakthrough on ImageNet proved the power of deep convolutional neural networks: **cut previous error rate in ~ half**
 - *Designed by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton*

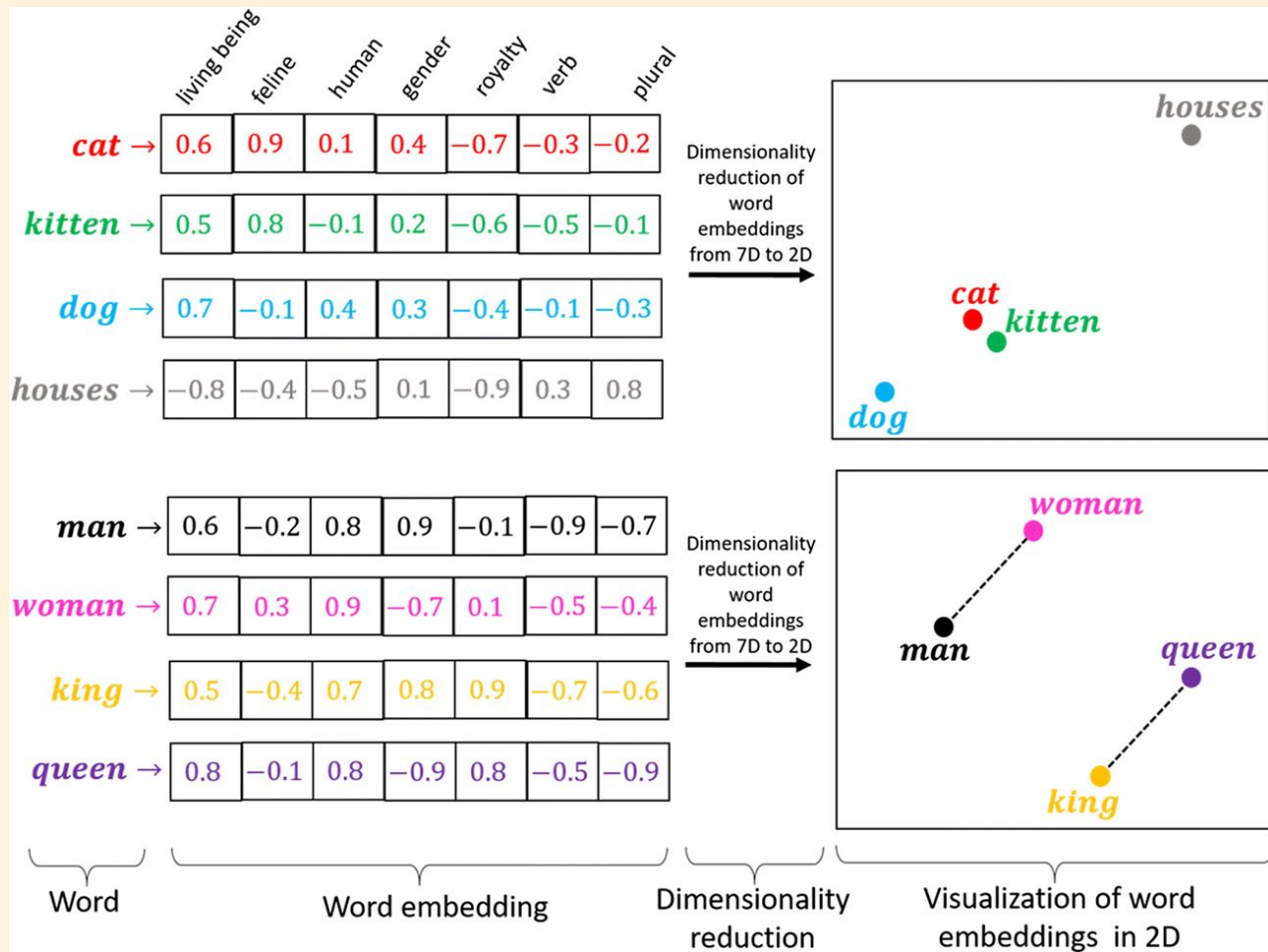


The Vector Revolution (2013)

The introduction of **Word2Vec** and **GloVe**. Instead of representing the word "Apple" as a unique ID, we represented it as a point in a 300-dimensional space.

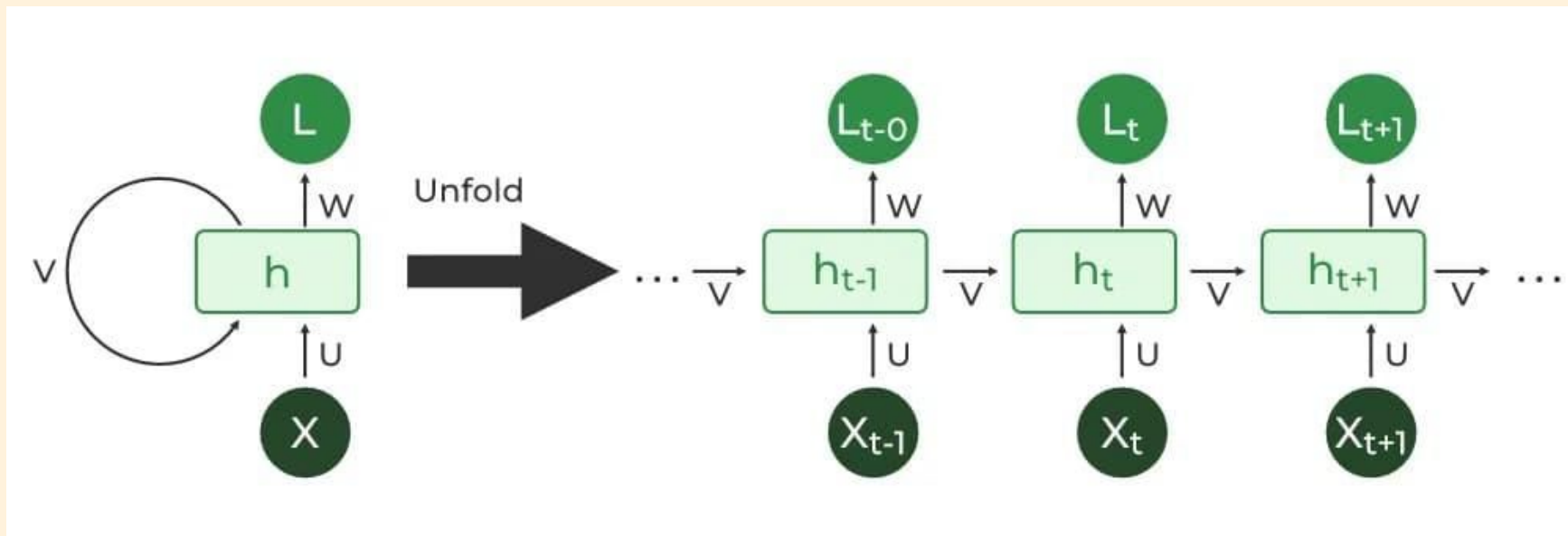
Key Insight: *"You shall know a word by the company it keeps."*

→ Words with similar meanings (e.g., "King" and "Queen") were mathematically closer together.



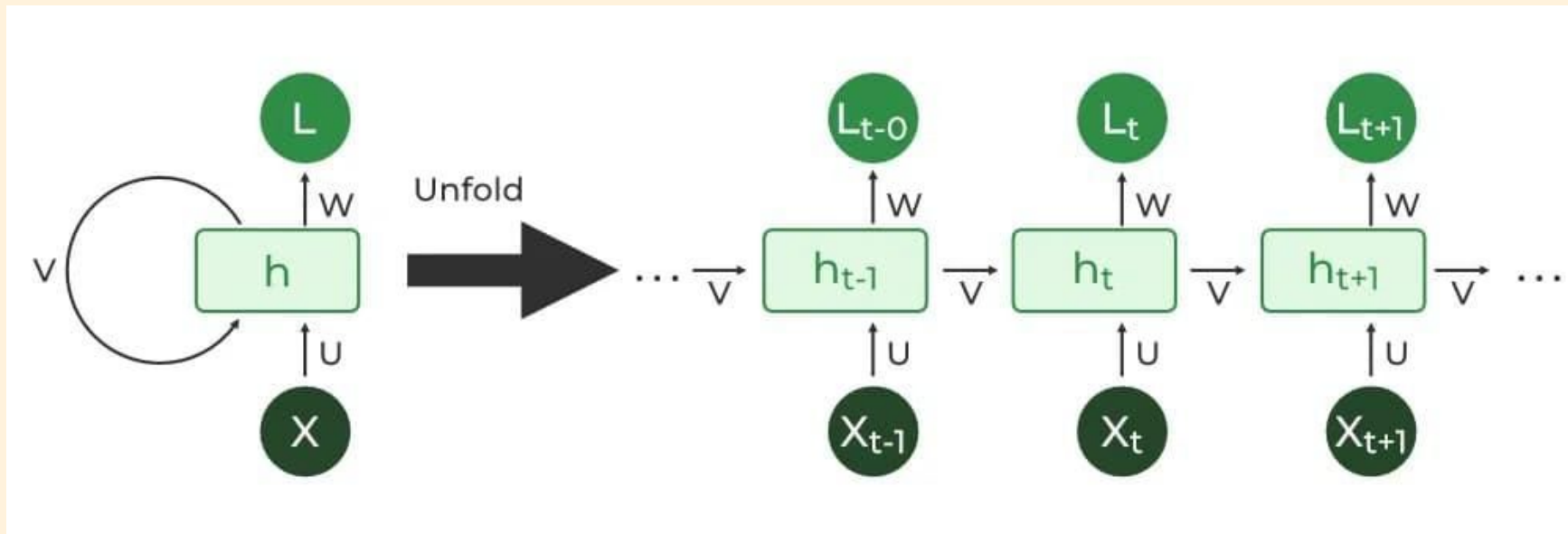
Recurrent Neural Networks (RNNs)

RNNs process tokens one by one, maintaining a "hidden state."



Recurrent Neural Networks (RNNs)

The Problem: **The "Vanishing Gradient."** As the input sentence grows, the influence of the first word on the last word decays to zero.



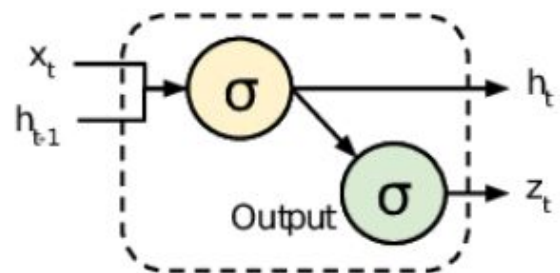
LSTMs (Long Short-Term Memory)

Introduced **gates** and a **cell state**

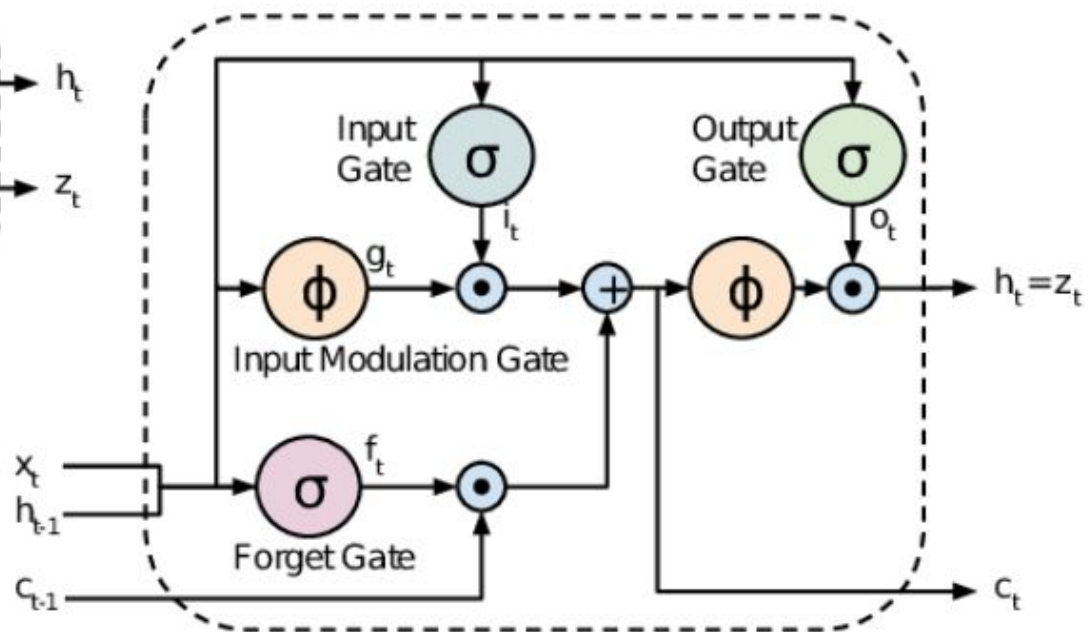
- Gates control the flow of information into and out of the cell state
- Input, Forget, and Output gates
- Cell state acts as a persistent memory conveyor belt.

Solved the vanishing gradient problem!

RNN Unit



LSTM Unit



LSTMs (Long Short-Term Memory)

New problem: Still **sequential**.

- You cannot parallelize the training.
- You must wait for word #1 to finish before starting word #2.

The Transformer & Scaling Laws (2017 – Present)

The paper "**Attention is All You Need**" removed the sequence requirement entirely.

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

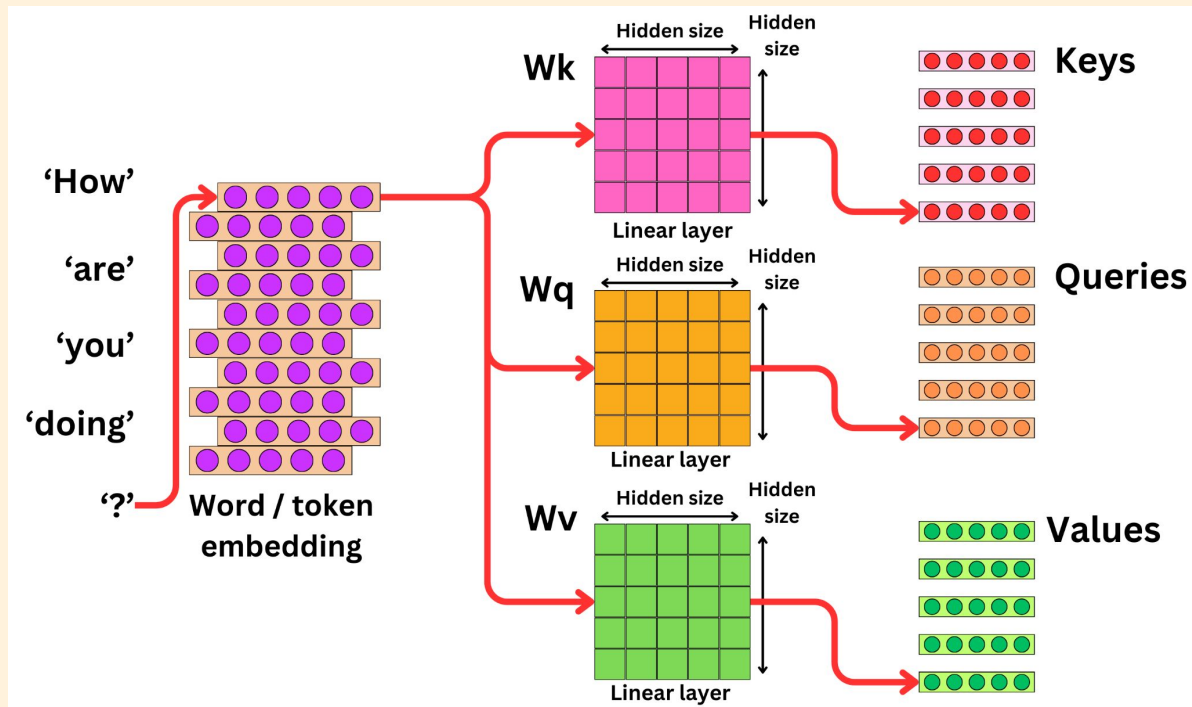
Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* †
illia.polosukhin@gmail.com

The Transformer

Self-Attention: Every word in a sentence looks at every other word *at the same time*. This allows the model to understand global context instantly.



Scaling Laws

Researchers discovered that as you increase the number of parameters and the amount of data, the "Loss" (error) decreases in a predictable power law. This led to the race for trillion-parameter models.



NVIDIA Blog

How Scaling Laws Drive Smarter, More Powerful AI

AI scaling laws describe how model performance improves as the size of training data, model parameters or computational resources increases.

Feb 12, 2025



LLMs today

#	Model ↑↓	LLM Stats [?] ↑↓	Reasoning [?] ↑↓	Coding [?] ↑↓
1	 GPT-6 Astra	59.9	57.9	48.5
2	 Claude Fable 5.1	55.9	53.0	35.5
3	 GPT-5.6 Sol	55.0	54.1	46.0
4	 Claude Mythos Preview UNRELEASED	54.9	55.8	44.8
5	 Claude Opus 5	54.9	53.9	41.9
6	 Muse Spark 1.3	54.4	51.8	41.7
7	 Claude Fable 5	54.3	51.8	45.7
8	 Kimi K3	53.1	51.8	43.3
9	 GLM-5.3	52.8	51.9	42.8
10	 DeepSeek-V4-Pro-0813	52.1	50.1	40.7

Today's Lecture

1. History of language modeling
2. **Defining the AI Agent**

From Completion to Agency

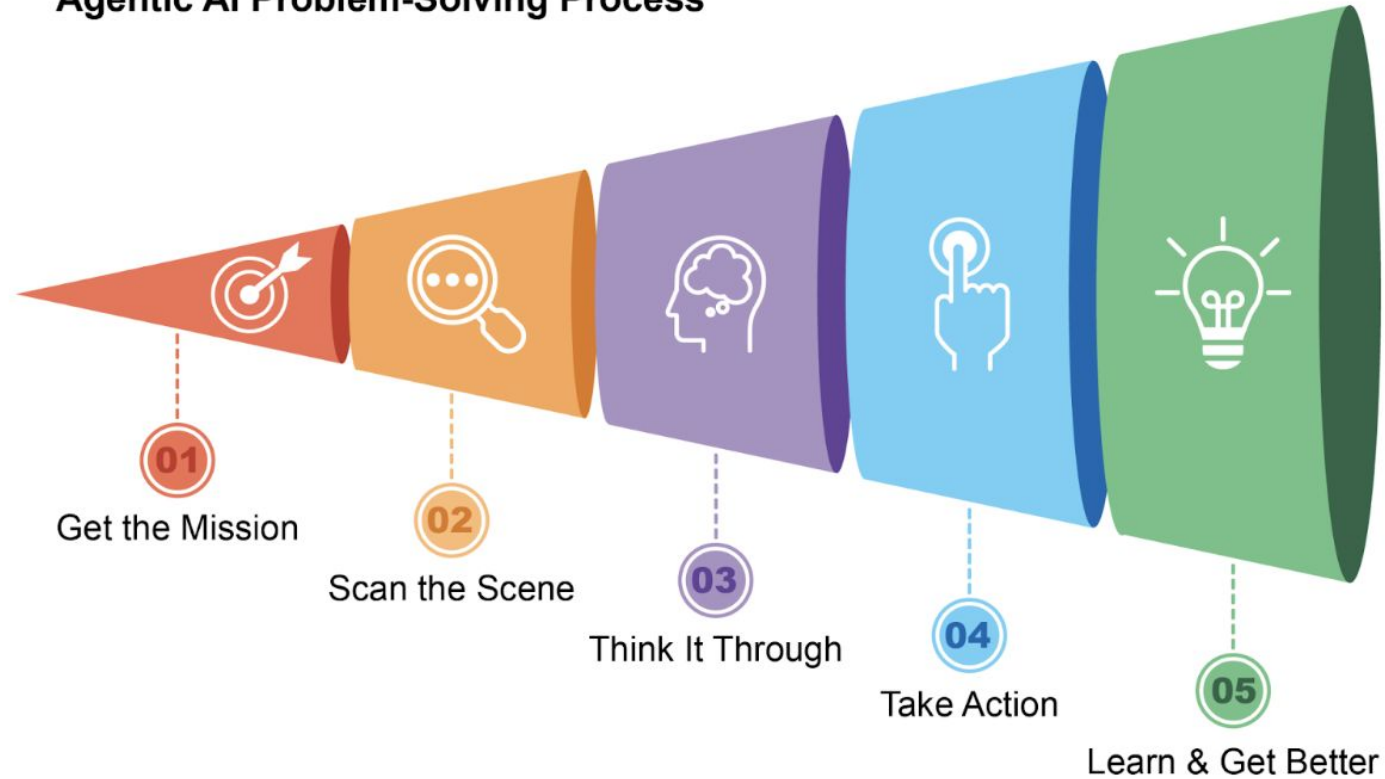
Passive LLM: You give it a prompt, and it gives you a completion. It has no memory of the world and no way to change it.

Active Agent: An agent is Autonomous, Reactive, and Proactive. It can observe a result, realize it made a mistake, and try a different approach.

Definition

An agent is a system that chooses and executes actions based on intermediate results.

Agentic AI Problem-Solving Process



The Cognitive Architecture of an Agent

1. Planning
2. Memory
3. Tool Use / Action
4. Perception



The Cognitive Architecture of an Agent

1. Planning

- a. Task Decomposition: The agent breaks a "Goal" into "Steps."
- b. Self-Reflection: The agent looks at its own output and asks, "Does this make sense?" (e.g., the Self-Refine or Reflexion frameworks).
- c. Chain of Thought (CoT): Forcing the model to "show its work" step-by-step, which significantly reduces logic errors.

The Cognitive Architecture of an Agent

1. Planning

1. **Memory**

- a. Short-term: The conversation history stored in the context window.
- b. Long-term: Vector Databases (using RAG or other retrieval mechanisms). The agent can "search" its past experiences or external documents to inform its current action.

The Cognitive Architecture of an Agent

1. Planning
2. Memory

1. Tool Use / Action

- a. Function Calling: The model doesn't just write text; it outputs a specific piece of code that a computer can execute.
- b. The MRKL (Miracle) Architecture: *Modular reasoning, knowledge, and language*.
 - i. A system that combines an LLM with "expert" tools (like a calculator or a weather API).

The Cognitive Architecture of an Agent

1. Planning
2. Memory
3. Tool Use / Action

1. Perception

- a. Modern agents are often Multimodal. They don't just "read" the internet; they can "see" a screenshot of a website or "hear" a user's voice command.
- b. Inputs from different modalities can be processed together to give the agent a better understanding of its task

The ReAct Pattern (Reasoning + Acting)

The most common agentic loop is ReAct:

Example: “How many women are there in Paris?”

Step 1 (Thought): "I need to find the population of Paris and divide it by 2."

Step 2 (Action): Search("Population of Paris")

Step 3 (Observation): "The search result says 2.1 million."

Step 4 (Thought): "Now I need to use the calculator tool."

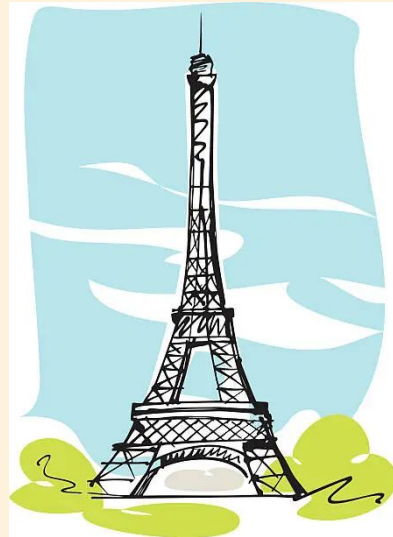
Step 5 (Action): Calculate($2100000 / 2$)

Activity: Applying ReAct (15 min)

Get into groups of 3-4. Pick an open-ended problem (e.g. plan a 1-week trip to Paris on a 500 euro budget)

Sketch out how an agent would approach this problem

1. **Planning:** How does the agent break this down?
2. **Tool Use:** Which APIs/external tools are needed?
3. **Memory:** What does it need to "retrieve" from the user's past data to succeed?
4. **Perception:** What real-time data must it "observe" to make a decision?



Announcements

- There are many people on the waitlist! If you plan to drop, please Imk sooner rather than later
- Come talk to me right now if you're unsure whether this class is a good fit for you
- Reminder to sign up for EdStem + Gradescope